

Class Test - 3

Unit - 1

S. Praveen.

192421252

B.Tech - IT

CSA0606.

16
100

- ① A CyberSecurity System analyzes threats using the recurrence $T(n) = T(n-1) + n \log n$. Apply the Substitution method to solve the recurrence. Expand step by step and derive the complexity. Analyze how the algorithm scales with increasing threat data. Interpret efficiency using asymptotic notation.
- ② A distributed database Processes solves the recurrence $T(n) = 5T(n/2) + n$. Apply the Master Theorem to determine the time complexity. Identify Parameters and justify the Case Selection. Analyze how this impacts Performance for large distributed systems. Interpret efficiency using asymptotic notation.
- ③ A fraud detection algorithm follows the recurrence $T(n) = 2T(n/2) + n^2 \log n$. Apply the Master Theorem to determine the time complexity. Compare $f(n)$ with $n^{\log_2 2}$ and justify the correct case. Analyze Performance for large transaction datasets. Interpret efficiency clearly.
- ④ A Sensor network algorithm divides a monitoring Problem into three equal subproblems and Performs only constant work at each level for control operations. Apply the Substitution Method to solve the Problem recurrence $T(n) = 3T(n/3) + 1$. Show all intermediate steps and derive the final asymptotic complexity. Analyze how this algorithm scales with increasing Sensors nodes. Interpret the efficiency using asymptotic notation.

$$4) T(n) = 3T\left(\frac{n}{3}\right) + T$$

$$T(1) = 1$$

$$T(n) = 3T\left(\frac{n}{3}\right) + 1$$

$$T(n) = 9T\left(\frac{n}{9}\right) + 3 + 1 \quad 3^2 T\left(\frac{n}{3^2}\right) + 3^1$$

$$T(n) = 9 \left[3T\left(\frac{n}{27}\right) + 9 + 3 + 1 \right]$$

$$T(n) = 27T\left(\frac{n}{27}\right) + 9 + 3 + 1 + 3^2 \left(\frac{n}{3^3}\right)$$

$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + \sum_{i=0}^{k-1} 3^i$$

$$\sum_{i=0}^{k-1} 3^i = 3\left(\frac{3^k - 1}{3 - 1}\right) = 3^k - 1$$

$$\frac{n}{3^k} = 1 \Rightarrow n = 3^k \Rightarrow k$$

~~$$T(n) = 3^k T\left(\frac{n}{3^k}\right) + \left(\frac{3^k - 1}{2}\right)$$~~

~~$$T(n) = \frac{1}{2}n + \frac{1}{2}n - \frac{1}{2}$$~~

~~$$T(n) = \frac{1}{2}n + \frac{1}{2}n - \frac{1}{2} \Rightarrow T(n) = \Theta(n)$$~~

Asymptotic complexity.

The domination term is approach insufficient it in
find Bound $T(n) = \log O(n)$ Limit Tree
complexity.

①

$$\begin{aligned}
 T(n) &= T(n-1) + \log n \\
 &= T(n-2) + \log(n-1) + \log n \\
 &= T(n-2) + \log(n-2) + \log(n-1) + \log n
 \end{aligned}$$

$$T(n) = T(1) = \sum_{i=2}^n \log i$$

$$\leq \log i = \log(n)$$

$$\log(n) = \Theta(\log n)$$

$$T(n) = \Theta(n \log n)$$

- Large Volume of large Security threat Cost.
- Interpolation of this algorithm is insufficient extremely

3)

$$T(n) = 2T(n/2) + n^2 \log n$$

$$a = 2, b = 2$$

$$n \log a = 2 = n$$

$$n^2 \log n > n$$

$$n \log b = n \log 2^2 = n'$$

compare $f(n) =$ with $n \log b^a$

$$f(n) = n^2 \log n = \Theta(n^{1+\epsilon}) \text{ where } \epsilon = 1$$

check $(c \cdot F)(n, b) \leq c F(n)$ for $c < 1$

$$\left(\frac{1}{2}\right)^2 \log\left(\frac{n}{2}\right) = \frac{1}{2} n^2 (\log n - \log 2) \leq \frac{1}{2} n^2 \log n$$

This Condition where $T_{tree} = c \frac{1}{2}$

* Justification Since $f(n)$ grow polynomially faster and Satisfied

Regularity

* Falls into case three Master Theorem.

2. $T(n) = 5T(n/2) + n$

$c=5$ $b=2$, $f(n)=n$

$n \log b^c = n \log_2 5$

$\log_2 5 = 2.32$

$n \log_2 5 = n^{2.32}$

$n < n^{2.32}$

$T_n = \Theta(n \log_2 5)$

$\Theta = [n^{2.32}]$

Justification Since $f(n)$ grow polynomially slower than falls into case 1 of Master Theorem.

Thus complexity is $\Theta(n \log n)$

$\Theta(n \log n)^{2.32}$

A complexity of $\approx \Theta(n)$ Scales Poorly for very large distributed systems.